

Euro Coin Image Classification

Machine Learning Programming Assignment

Name: Yasamin Hosseinzadeh Sani

Student ID: 531672

June 2026

Abstract

This work develops classifiers for eight Euro coin denominations using a balanced dataset of 800 training and 160 test images. Logistic Regression, a custom convolutional neural network (CNN), and transfer learning with MobileNetV2 were compared. Models were evaluated using both classification accuracy and the mean absolute monetary value of misclassification. Standard MobileNetV2 provided the best overall trade-off, achieving 78.12% test accuracy, a mean monetary error of 2.19 cents, and a maximum error of 40 cents. A cost-sensitive MobileNetV2 decision rule achieved the highest accuracy, 78.75%, but a slightly worse mean monetary error of 2.52 cents.

1 Problem and Data Analysis

The task is to classify images into eight classes: 1, 2, 5, 10, 20, and 50 cents, plus 1 and 2 Euro coins. Each class contains 100 training and 20 test images. All images are RGB PNG files of size 64×64 pixels. The classes are therefore perfectly balanced.

The images contain substantial variation in illumination, sharpness, background, orientation, and visible coin side. Coins may be rotated in any direction, while several denominations share similar colour and circular geometry. As a result, raw intensity values alone are not sufficient: a robust classifier should learn local edges, textures, relief patterns, and colour distributions, while remaining relatively invariant to rotation, translation, and moderate illumination changes. The brightness boxplot also shows substantial within-class spread, meaning that illumination is not a stable class-specific cue. This is important because a classifier that over-relies on global brightness could perform well on a limited subset of images but fail under different acquisition conditions. These observations motivate normalisation, data augmentation, and models that preserve spatial structure.



Figure 1: Representative training image from each Euro coin class.

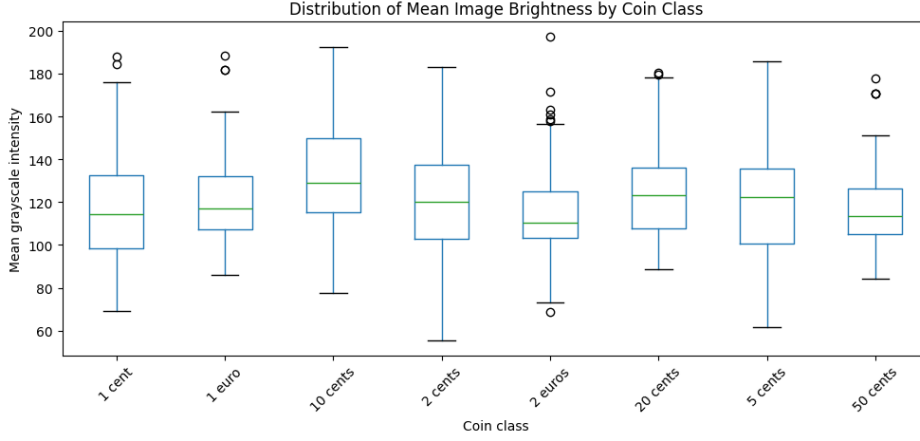


Figure 2: Distribution of mean image brightness across the eight coin classes.

The 800 supplied training images were divided using a stratified 80/20 split, giving 640 training and 160 validation images while preserving the same class proportions in both subsets. The separate 160-image test set was kept untouched during model selection and hyperparameter tuning, and was used only once for final evaluation. This separation reduces the risk of optimistic performance estimates.

2 Methods

2.1 Evaluation Criteria

The standard classification metric was test accuracy. Precision, recall, F1-score, and confusion matrices were also inspected in order to identify whether errors were concentrated in particular denominations. Accuracy is appropriate because the dataset is balanced, but it treats all mistakes as equally severe. In this application, however, confusing 1 cent with 2 cents is less costly than confusing 20 cents with 2 Euros. Since classes represent different monetary values, the practical cost of an error was measured as

$$C(y, \hat{y}) = |v_y - v_{\hat{y}}|, \quad (1)$$

where v_y and $v_{\hat{y}}$ are the true and predicted coin values in cents. The reported monetary metric is

$$\text{MAE}_{\text{money}} = \frac{1}{N} \sum_{i=1}^N |v_{y_i} - v_{\hat{y}_i}|. \quad (2)$$

2.2 Logistic Regression Baseline

Each image was scaled to $[0, 1]$ and flattened into $64 \times 64 \times 3 = 12,288$ features. A pipeline containing **StandardScaler** and multinomial Logistic Regression was trained. Standardisation places all pixel features on a comparable scale and improves numerical stability during optimisation. This model provides an interpretable baseline, but flattening destroys the two-dimensional neighbourhood relationships that are important for recognising rims, digits, engravings, and bimetallic patterns. Consequently, the classifier can use colour and absolute pixel position, but it cannot explicitly learn translation-tolerant local motifs. This limitation is particularly relevant because the same coin may appear rotated or slightly shifted between images.

A cost-sensitive prediction rule was additionally applied to the model probabilities:

$$\hat{y} = \arg \min_j \sum_{k=1}^8 P(y = k | x) C(k, j). \quad (3)$$

This satisfies the requirement for an additional model designed to minimise the monetary value of misclassification. Unlike the usual maximum-probability rule, this decision rule may deliberately select a slightly less probable class when that prediction has a lower expected monetary penalty.

2.3 Custom CNN

The custom CNN contained three convolutional blocks, each using convolution, batch normalisation, ReLU, and max pooling. Convolutional filters learn local visual patterns, batch normalisation stabilises intermediate activations,

and pooling progressively reduces spatial resolution while retaining dominant features. The classifier used a fully connected layer and dropout to reduce overfitting. Training used cross-entropy loss, Adam, weight decay, learning-rate reduction, checkpointing, and early stopping. Augmentation included rotation, translation, scaling, and colour jitter so that the model would not rely too strongly on a single orientation or lighting condition. Despite this architectural advantage, the custom CNN was trained from only 640 images. Its relatively low test accuracy suggests that learning robust visual features from scratch was difficult under this data regime. Nevertheless, its lower monetary error indicates that many of its mistakes occurred between nearby denominations rather than between very different coin values.

2.4 Transfer Learning with MobileNetV2

MobileNetV2 pretrained on ImageNet was used as the strongest classifier. Images were resized to 160×160 and normalised with ImageNet statistics so that their input distribution matched the one used during pretraining. MobileNetV2 uses depthwise separable convolutions and inverted residual blocks, providing strong visual representations with relatively low computational cost. The pretrained filters already encode generic edges, textures, contours, and object-part patterns, which is advantageous when the target dataset is too small to learn all such representations from scratch. Training had two stages:

1. freeze the feature extractor and train a new eight-class head;
2. unfreeze the final four feature blocks and fine-tune them with a smaller learning rate.

Moderate rotation, translation, scaling, brightness, contrast, and saturation augmentation were applied during training. These transformations reflect the variability observed in the exploratory analysis without introducing excessively unrealistic samples. The best checkpoint was selected by validation loss rather than training accuracy, because validation loss also reflects prediction confidence and is more sensitive to overfitting. The best fine-tuning checkpoint occurred at epoch 13, with validation loss 0.5545 and validation accuracy 78.75%.

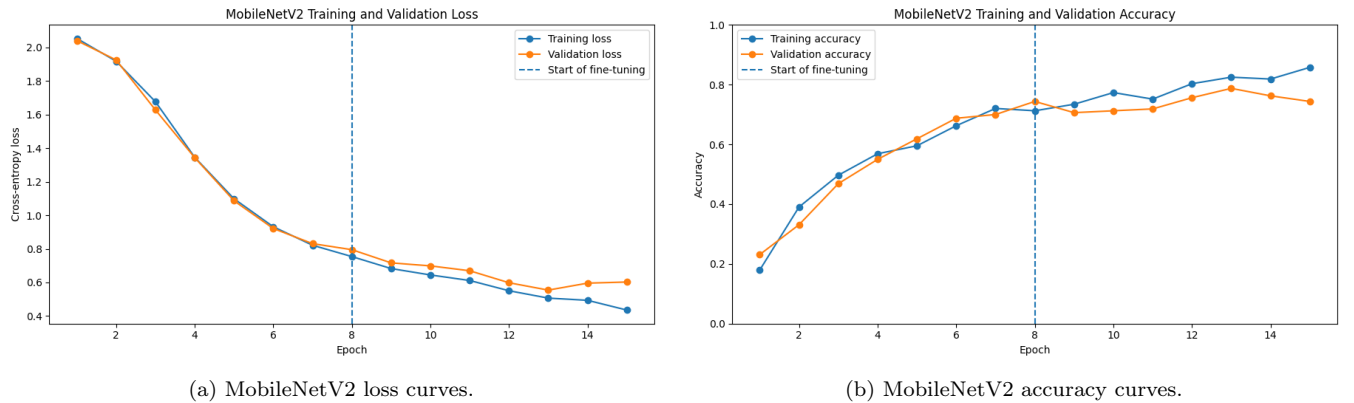


Figure 3: Training head for eight epochs, followed by fine-tuning.

The learning curves show a clear improvement during the frozen-head stage and a further gain after fine-tuning begins. Training and validation accuracy remain relatively close for most epochs, which suggests that the model did not severely overfit. After epoch 13, training accuracy continues to increase while validation performance starts to fluctuate, supporting the use of validation-loss checkpointing and early stopping.

3 Results and Model Behaviour

Table 1: Performance on the independent 160-image test set.

Method	Accuracy	Mean cost (cents)	Max cost (cents)
Logistic Regression	68.13%	8.16	180
Cost-sensitive Logistic Regression	68.75%	7.12	180
Custom CNN	61.25%	5.01	48
Cost-sensitive Custom CNN	58.13%	5.67	95
MobileNetV2	78.12%	2.19	40
Cost-sensitive MobileNetV2	78.75%	2.52	40

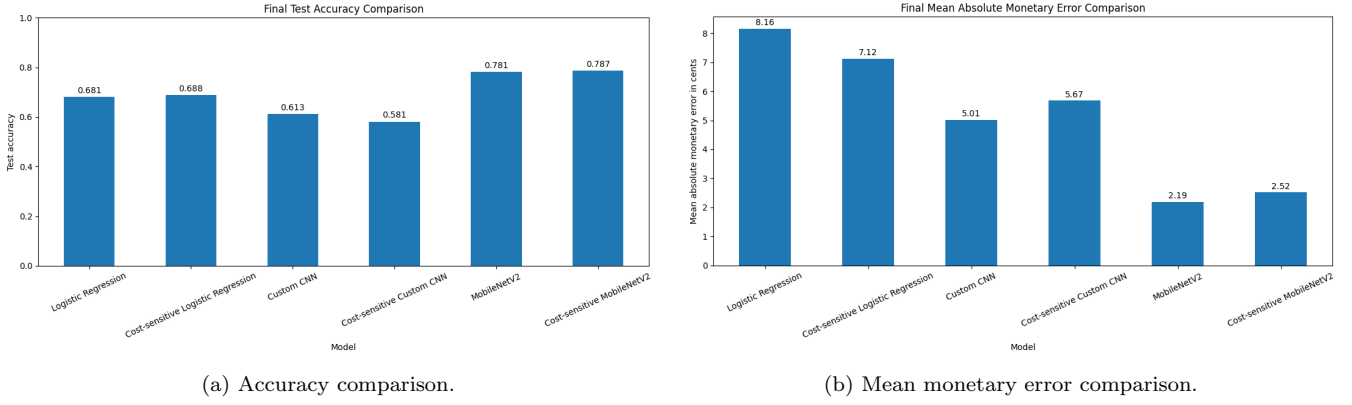


Figure 4: Final comparison of all six prediction strategies.

MobileNetV2 improved accuracy by 10.00 percentage points over the Logistic Regression baseline and reduced mean monetary error by approximately 73.1%. Its confusion matrix shows perfect recognition of the 1 Euro and 2 Euro classes, while most remaining errors occur among visually similar low-value cent coins. In particular, the confusions are concentrated among copper-coloured 1, 2, and 5 cent coins and among the gold-coloured 10, 20, and 50 cent coins. This pattern is consistent with the fact that coins within each material group share similar colour and global shape. Importantly, the largest monetary error was limited to 40 cents, showing that the remaining mistakes were generally between economically closer denominations. Compared with Logistic Regression, MobileNetV2 therefore improved not only the number of correct predictions but also the severity profile of the errors. This is practically important because a classifier with similar accuracy could still be undesirable if its few mistakes involved large value differences.

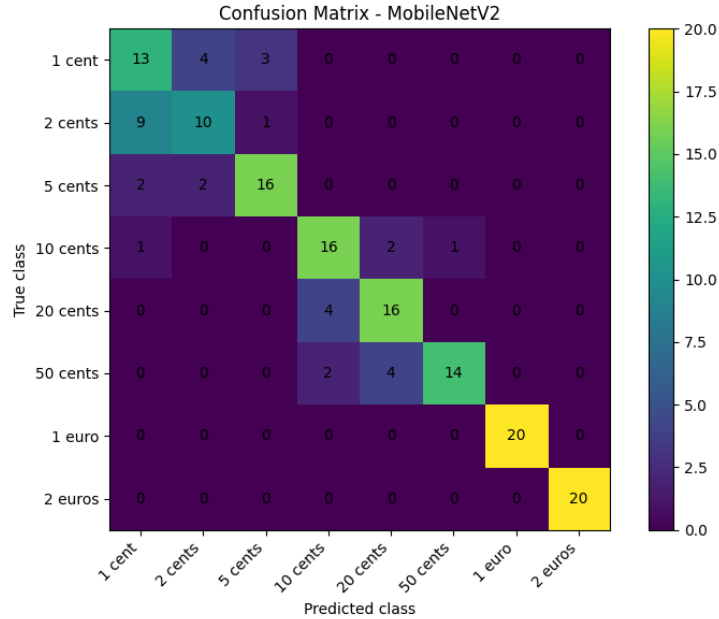


Figure 5: Confusion matrix for standard MobileNetV2.

The cost-sensitive MobileNetV2 classified one additional image correctly, but its average monetary error increased from 2.19 to 2.52 cents. Therefore, the expected-cost decision rule did not improve the monetary objective for this model. A plausible explanation is that the softmax probabilities were not perfectly calibrated: if confidence values do not accurately represent true class likelihoods, expected-cost minimisation can shift some predictions in an undesirable direction. Standard MobileNetV2 was therefore selected as the preferred overall model because it achieved the lowest average monetary cost while retaining nearly the highest accuracy.

The comparison between the custom CNN and Logistic Regression further illustrates the trade-off between the two objectives. Logistic Regression achieved higher accuracy, but the custom CNN had a lower mean and maximum monetary error. This means that the CNN made more mistakes overall, yet its mistakes were less damaging on average. Therefore, reporting only accuracy would hide an important aspect of model behaviour.

4 Limitations and Possible Improvements

The test set contains only 160 images, so small differences such as 78.12% versus 78.75% correspond to only one image and should not be over-interpreted. In addition, the images are low-resolution and may contain background information that is unrelated to the coin itself. A dedicated coin segmentation or centre-cropping stage could reduce this source of variation. Further improvements could include probability calibration before cost-sensitive prediction, cross-validation on the training set, stronger but realistic rotation augmentation, and evaluation on images collected from new cameras or environments.

5 Reproducibility

The experiments were implemented in Python 3.12 using NumPy, pandas, Pillow, Matplotlib, scikit-learn, PyTorch, and TorchVision. The random seed was fixed to 42.

To reproduce the results, create and activate a virtual environment, install the dependencies from `requirements.txt`, and run the notebook `notebooks/euro_coin_classification.ipynb` from top to bottom.

The dataset must follow this directory structure:

```
data/train/{001,002,005,010,020,050,100,200}
data/test/{001,002,005,010,020,050,100,200}
```

The training notebook saves model checkpoints under `outputs/models/`, figures under `outputs/figures/`, and numerical results under `outputs/results/`.

6 Use of AI Tools

AI tools were used for:

- explaining assignment requirements and planning the workflow;
- helping structure, edit, and format this report in L^AT_EX.

All experiments were executed on the provided dataset, and the reported numerical results were obtained from the submitted notebook.

7 Conclusion

The project analysed the dataset, trained and evaluated multiple classifiers, visualised model behaviour, and explicitly considered the monetary cost of errors. The comparison shows that model quality cannot be judged by accuracy alone: the custom CNN had lower accuracy than Logistic Regression but produced less costly mistakes, whereas MobileNetV2 improved both objectives. Transfer learning substantially outperformed both raw-pixel Logistic Regression and a CNN trained from scratch, confirming its suitability for small image datasets. Standard MobileNetV2 was the best overall model, combining 78.12% accuracy with the lowest mean monetary error of 2.19 cents. Cost-sensitive MobileNetV2 achieved the highest accuracy, 78.75%, but did not improve monetary cost. Further improvement could be obtained through probability calibration, larger datasets, or coin-specific augmentation and segmentation. Overall, the results show that transfer learning is the most effective strategy for this small image dataset, while cost-aware evaluation provides a more complete assessment than accuracy alone.

“I affirm that this report is the result of my own work and that I did not share any part of it with anyone else except the teacher.”